



# Phase 3: Intelligent Model Selection & Training

---

---

## CONTEXT — What this phase is and why it matters

---

Intelligent model selection is the most important decision in an ML project. The model must be chosen based on:

- Dataset characteristics (size, type, distribution)
- Problem type (binary classification)
- Interpretability requirements
- Computational constraints

Models should not be limited; we evaluate from the simplest to the most advanced.

---

---

## INPUTS — Required input files

---

- 1\_Data/Processed/encoded\_dataset.csv
- Phase2\_Report.md

### Simple models (linear)

| Model               | Pros                              | Cons                                     | Best for                           |
|---------------------|-----------------------------------|------------------------------------------|------------------------------------|
| Logistic Regression | Interpretable, fast, calibrated   | Assumes linear separability              | Baseline + clinical interpretation |
| Naive Bayes         | Very fast, works with little data | Strong independence assumption           | Quick baseline                     |
| KNN                 | Non-parametric, simple            | Slow on large data, sensitive to scaling | Small datasets                     |

## Advanced models (tree-based)

| Model         | Pros                          | Cons                        | Best for                   |
|---------------|-------------------------------|-----------------------------|----------------------------|
| Random Forest | Robust, handles non-linearity | Less interpretable          | Tabular data baseline      |
| XGBoost       | State-of-the-art on tabular   | Many hyperparameters        | Performance-driven tasks   |
| LightGBM      | Fast, memory-efficient        | Sensitive to small datasets | Large tabular datasets     |
| CatBoost      | Handles categoricals natively | Slower training             | Categorical-heavy datasets |

## Deep-learning models

| Model  | Pros                    | Cons                                   | Best for                          |
|--------|-------------------------|----------------------------------------|-----------------------------------|
| MLP    | Flexible, high capacity | Needs lots of data, less interpretable | Large datasets                    |
| TabNet | Tabular-aware attention | Complex tuning                         | When interpretability + DL matter |

---



---

## TASKS — What needs to be done

---



---



### TASK 1: Dataset analysis for model selection

---

```
def dataset_analysis_for_models(df, target_col):
    """Dataset analysis to drive model-selection decisions."""
    n_samples = len(df)
    n_features = len(df.columns) - 1
    class_balance = df[target_col].value_counts(normalize=True).to_dict()

    rec = []
    if n_samples < 1000:
        rec.append("Prefer simple models (LR, RF) – limited data")
    if n_samples >= 1000:
        rec.append("Tree ensembles (XGBoost, LightGBM, CatBoost) recommended")
    if n_samples >= 10000:
        rec.append("Deep-learning models (MLP, TabNet) viable")
    if min(class_balance.values()) < 0.3:
        rec.append("Class imbalance detected – apply SMOTE / class-weights")

    return {
        'n_samples': n_samples,
        'n_features': n_features,
        'class_balance': class_balance,
        'recommendations': rec,
    }
```

---

## TASK 2: Data preparation for modeling

---

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

def prepare_data(df, target_col, test_size=0.2, random_state=42):
    """Train/test split (80/20 stratified) with StandardScaler fit on train only."""
    X = df.drop(columns=[target_col])
    y = df[target_col]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, stratify=y, random_state=random_state
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    return X_train_scaled, X_test_scaled, y_train, y_test, scaler
```

---

## TASK 3: Class-imbalance handling

---

```

from imblearn.over_sampling import SMOTE, ADASYN

def handle_imbalance(X_train, y_train, method='smote'):
    """Class-imbalance handling. Always applied to training data only."""
    if method == 'smote':
        sm = SMOTE(random_state=42)
    elif method == 'adasyn':
        sm = ADASYN(random_state=42)
    else:
        return X_train, y_train
    return sm.fit_resample(X_train, y_train)

```

For models that support `class_weight`, pass `class_weight='balanced'` instead of resampling when appropriate.

---

## TASK 4: Define models

---

```

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
import xgboost as xgb
import lightgbm as lgb
from catboost import CatBoostClassifier

def define_models(random_state=42):
    """Define the candidate model roster with scientific justification."""
    return {
        'LogisticRegression': LogisticRegression(max_iter=1000, random_state=random_state),
        'DecisionTree': DecisionTreeClassifier(random_state=random_state),
        'RandomForest': RandomForestClassifier(random_state=random_state),
        'GradientBoosting': GradientBoostingClassifier(random_state=random_state),
        'XGBoost': xgb.XGBClassifier(random_state=random_state, eval_metric='logloss'),
        'LightGBM': lgb.LGBMClassifier(random_state=random_state),
        'CatBoost': CatBoostClassifier(random_seed=random_state, verbose=0),
        'SVM': SVC(probability=True, random_state=random_state),
        'KNN': KNeighborsClassifier(),
        'NaiveBayes': GaussianNB(),
    }

```

---

## TASK 5: Hyperparameter tuning with Optuna

---

```

import optuna

def tune_lightgbm(X, y, n_trials=50):
    """Hyperparameter tuning for LightGBM with Optuna."""
    def objective(trial):
        params = {
            'num_leaves': trial.suggest_int('num_leaves', 16, 128),
            'max_depth': trial.suggest_int('max_depth', 3, 12),
            'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True)
        },
        'n_estimators': trial.suggest_int('n_estimators', 100, 1000),
    }
        model = lgb.LGBMClassifier(**params, random_state=42)
        score = cross_val_score(model, X, y, cv=5, scoring='roc_auc').mean()
        return score

    study = optuna.create_study(direction='maximize')
    study.optimize(objective, n_trials=n_trials)
    return study.best_params

```

Apply Bayesian optimization to LR, RF, XGBoost, LightGBM, CatBoost, and SVM.

---

## TASK 6: Train and evaluate models

---

```

from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, matthews_corrcoef)

def train_and_evaluate(models, X_train, y_train, X_test, y_test):
    """Train and evaluate all models. Reports AUC, accuracy, precision, recall, F1,
    MCC."""
    results = []
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        y_proba = model.predict_proba(X_test)[:, 1]
        results.append({
            'model': name,
            'accuracy': accuracy_score(y_test, y_pred),
            'precision': precision_score(y_test, y_pred),
            'recall': recall_score(y_test, y_pred),
            'f1': f1_score(y_test, y_pred),
            'auc': roc_auc_score(y_test, y_proba),
            'mcc': matthews_corrcoef(y_test, y_pred),
        })
    return pd.DataFrame(results).sort_values('auc', ascending=False)

```

---

## TASK 7: Ensemble methods

---

```

from sklearn.ensemble import VotingClassifier, StackingClassifier

def build_voting_ensemble(top_models):
    """Voting classifier from the top-performing base models."""
    return VotingClassifier(estimators=top_models, voting='soft')

def build_stacking_ensemble(top_models, meta_model):
    """Stacking classifier with a meta-learner (e.g., logistic regression)."""
    return StackingClassifier(estimators=top_models, final_estimator=meta_model, cv=5)

```

Evaluate Voting and Stacking ensembles against the best single model.

---

## TASK 8: Deep learning (optional)

---

For larger datasets ( $n \geq 10,000$ ), evaluate MLPs and TabNet. For our cohort ( $n \approx 2,450$ ), the MLP is reported alongside trees as a reference.

---

## INTELLIGENT DECISIONS

---

The agent automatically decides:

- Whether SMOTE, ADASYN, class weights, or none should be used given the imbalance
  - Which models to include in the candidate roster based on dataset size
  - Whether to enable Optuna tuning or use sensible defaults given compute budget
  - Which ensembles to build (voting vs stacking) based on top-model diversity
- 

---

## OUTPUTS — Outputs of this phase

---

- 3\_Results/Phase3\_Models/Phase3\_Report.md
  - model\_comparison.csv — all metrics for all candidate models
  - hyperparameter\_results.csv — Optuna tuning logs and best params
  - 6\_Models/Saved/\*.joblib — trained model artifacts
  - Figures: ROC curves, PR curves, confusion matrices for each model (PNG + TIFF)
- 

---

## OUTPUT FORMAT — Report format

---

## Executive Summary

---

Best model, runner-ups, and headline metrics on the held-out test set.

### 1. Dataset Profile

---

Sample size, feature count, class balance, and data-driven model recommendations.

### 2. Models Evaluated

---

Roster of models with the rationale for inclusion.

### 3. Hyperparameter Tuning

---

Best hyperparameters per model with Optuna trial counts.

### 4. Performance Comparison

---

Table of AUC, accuracy, precision, recall, F1, MCC for all models on the test set.

### 5. Ensembles

---

Voting and stacking results vs the best single model.

### 6. Final Recommendation

---

Selected model(s) for downstream validation (Phase 4) and XAI (Phase 5), with justification.